Design Pattern per la Stabilità e Reactive

Stabilità

La stabilità, nel contesto dei sistemi distribuiti, coincide con il concetto di resilienza: un sistema resiliente è in grado di **continuare a elaborare le transazioni persino quando ci sono guasti** nei componenti che ne disturbano l'elaborazione. Per inquadrare il concetto, definiamo una "transazione" come un'unità di lavoro (ad esempio, "Un cliente fa un ordine"), la quale include l'esecuzione di molto codice, aggiornamenti di database e l'integrazione con sistemi esterni, appoggiandosi su un insieme di hardware, applicazioni e servizi. Pertanto, la stabilità non significa semplicemente che i server o le applicazioni sono "accesi" e in esecuzione, ma si traduce nel fatto pratico che **l'utente può effettivamente svolgere del lavoro** completando le transazioni nonostante le anomalie tecniche sottostanti.

Usa timeout

Il pattern "Usa timeout" serve a gestire i fallimenti di rete o di dispositivi, i quali possono smettere di funzionare in qualsiasi momento. Di fronte a questi eventi, il codice non può aspettare indefinitamente una risposta che potrebbe non arrivare mai, ma prima o poi deve rinunciare.

Intento e vantaggi

Il timeout è un meccanismo che impone di smettere di aspettare quando si sospetta che la risposta non arriverà. L'applicazione di timeout (specialmente nei punti di integrazione) è fondamentale per isolare i guasti, evitando che un problema in un sottosistema si propaghi bloccando i thread e causando fallimenti a cascata. Questo permette all'applicazione di andare comunque avanti.

Uso locale

I timeout sono cruciali anche all'interno di una singola applicazione, ad esempio per la gestione dei pool di risorse. Se le risorse si esauriscono, un thread in attesa dovrebbe essere sbloccato dallo scadere di un timeout, sia nel caso in cui la risorsa torni disponibile, sia in caso contrario. A livello di programmazione in Java, si suggerisce di usare sempre le varianti dei metodi che accettano un parametro di timeout, come wait(), poll(), offer() e tryLock().

Design del codice

Poiché la gestione dei timeout (*error-handling*) rischia di invadere e sporcare il codice funzionale dell'applicazione, il pattern consiglia di organizzare le interazioni a rischio in primitive riutilizzabili (creare un *QueryObject*) o in classi dedicate (come un generico *Gateway*, per la gestione di connessioni, errori, query e organizzazione dei risultati). Inserire le interazioni in un'unica classe semplifica enormemente anche la futura applicazione del design pattern *Circuit Breaker*.

Gestione dei retry (ritentativi)

Quando un'operazione fallisce per un timeout, spesso si cerca di ripeterla. Tuttavia, riprovare immediatamente è sconsigliato, poiché se il problema è transitorio l'operazione fallirà di nuovo. È preferibile invece accodare la richiesta e riprovare dopo un pò di tempo.

Interruttore di circuito

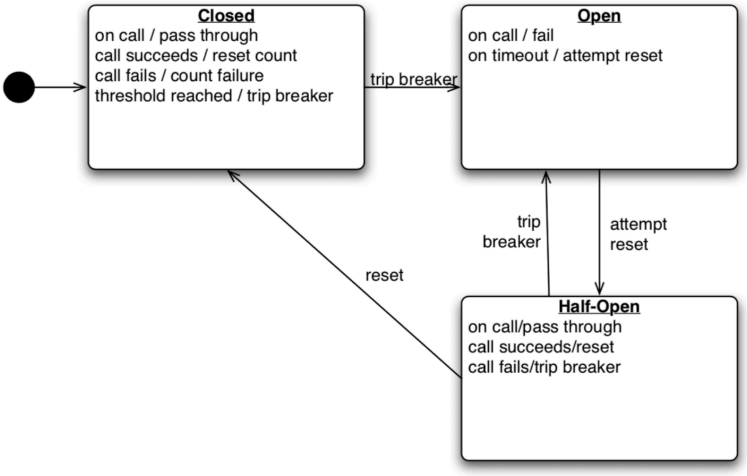
Il pattern si ispira al funzionamento dei fusibili nei circuiti elettrici: così come un fusibile si brucia per interrompere un sovraccarico di corrente ed evitare che un edificio prenda fuoco, il Circuit Breaker permette a un sottosistema software di fallire senza causare la distruzione (o il blocco) dell'intero sistema.

A livello software, questo componente "scherma" un'operazione pericolosa (come la chiamata ad un servizio esterno) evitando che vengano effettuate ulteriori chiamate quando si sa che il sistema di destinazione non funziona bene. È importante notare che non si tratta di un meccanismo per riprovare un'operazione, ma piuttosto per evitarne del tutto l'esecuzione temporaneamente.

Funzionamento e stati del circuito

Il ciclo di vita dell'interruttore di circuito si basa su tre stati principali:

1. **Chiuso** - stato normale:
    - l'interruttore fa passare regolarmente le chiamate.
    - se la chiamata ha successo, il sistema continua a funzionare normalmente.
    - se la chiamata fallisce (o va in timeout), il Circuit Breaker ne prende nota. Quando il numero (o la frequenza) di questi fallimenti supera una certa soglia prestabilita, l'interruttore "scatta" e passa allo stato Aperto.
2. **Aperto** - stato di blocco:
    - il circuito è interrotto. Tutte le chiamate rivolte al servizio falliscono immediatamente in modo preventivo, senza alcun tentativo di eseguire realmente l'operazione sul sistema remoto.
3. **Semi-aperto** - stato di verifica:
    - dopo che è trascorso un certo lasso di tempo, il Circuit Breaker ipotizza che il problema possa essersi risolto e passa allo stato Semi-aperto.
    - in questo stato, viene fatta passare la successiva chiamata (o una piccola percentuale di chiamate) per testare il servizio a rischio.
    - se questa chiamata ha successo, l'interruttore si resetta e torna allo stato Chiuso.
    - se invece la chiamata fallisce nuovamente, l'interruttore torna allo stato Aperto e si attende lo scadere di un nuovo timeout prima di riprovare.



Vantaggi e considerazioni

- **Degrado controllato**: il pattern è un modo efficace per degradare le funzionalità dell'applicazione quando un sistema è sovraccarico (ad esempio, intraprendendo azioni alternative se il servizio di verifica della carta di credito non risponde, scelta che coinvolge il cliente).
- **Prevenzione dai fallimenti a cascata**: è il pattern fondamentale per proteggere i punti di interazione del sistema, evitando che risposte lente o guasti si propaghino bloccando i thread dell'applicazione chiamante.
- **Sinergia con il Timeout**: lavora in combinazione con il pattern Timeout. Il timeout è infatti l'evento che indica all'interruttore che c'è un problema di integrazione.
- **Monitoraggio** (logging): l'entrata in azione di un Circuit Breaker indica un problema significativo. Per questo motivo, ogni cambiamento del suo stato deve essere registrato nei log e costantemente monitorato per analizzare i trend.

Bulkheads (paratìe)

Il pattern Bulkheads prende ispirazione dalle paratie delle navi, ovvero quelle barriere che dividono lo scafo in compartimenti a tenuta stagna. In questo modo, se c'è una falla, l'acqua non allaga l'intera nave facendola affondare. Nel software, questo si traduce nel principio di contenimento del danno: partizionando un sistema, si evita che un guasto o un'anomalia in una singola parte si propaghi distruggendo l'intero sistema. In un ambiente orientato ai servizi, questo evita che un'applicazione vada giù a causa di una catena di reazioni.

Modalità di implementazione e granularità

Le paratie possono essere applicate a diversi livelli di granularità:

- **Ridondanza fisica** (larga scala): è la forma più comune. Consiste nell'avere server indipendenti, in modo che il guasto hardware di uno non intacchi gli altri. Si possono avere intere farm di server dedicate esclusivamente a operazioni *mission-critical* (es. server dedicati al check-in isolati da quelli che gestiscono le query sui voli, in modo che un picco di richieste per i voli non blocchi i check-in).
- **Virtualizzazione**: offre il vantaggio dell'efficienza (condividendo le risorse hardware fisiche) unito alla sicurezza delle paratie. Si usano server virtuali dedicati la cui capacità (*ogni partizione ha la sua riserva di capacità*) può essere modificata a livello amministrativo (es. facendo il boot di nuove macchine virtuali o migrandole altrove). Questo partizionamento fa sì che la capacità totale necessaria sia minore rispetto alla somma delle singole capacità di picco (*se tutti i server sono condivisi, sarà necessaria meno capacità totale*).
- **CPU binding** (piccola scala): legare (binding) un processo a una specifica CPU è una forma di paratia che evita che un processo "fuori controllo" monopolizzi tutti i cicli di calcolo bloccando l'intera macchina.
- **Thread pool**: le paratie possono essere create anche a livello applicativo separando i pool di thread.

Il grande vantaggio di questo pattern è che permette di **preservare una parte di funzionalità** quando si verifica un problema critico.

Sistemi reattivi

I sistemi reattivi sono sistemi progettati per esibire due caratteristiche fondamentali: resilienza e messaggistica.

- **Resilienza**: un sistema reattivo garantisce di rispondere in tempi appropriati sebbene in presenza di guasti. Questa capacità viene ottenuta applicando concetti come repliche, contenimento, isolamento e delega. Fondamentalmente, i guasti vengono isolati all'interno di un componente specifico: se una parte si blocca e si riavvia, questo evento non compromette l'intero sistema e il client chiamante non viene sovraccaricato dalla necessità di dover gestire i fallimenti del componente.
- **Messaggistica**: per connettere i vari componenti, i sistemi reattivi si affidano ad un sistema di passaggio di messaggi asincroni. L'uso dei messaggi assicura tre vantaggi architetturali chiave:
  - **lasco accoppiamento** (bassa dipendenza) tra i componenti.
  - ulteriore **isolamento**.
  - **location transparency** (trasparenza rispetto alla posizione fisica dei servizi sulla rete).